14.6 Summary

- **Graph**: A data structure that can model many problems, in general works well when there are entities connected by some kind of proximity relation: it could indifferently be cities and roads, or social network users connected through virtual friendship.
- **Representation**: While usually the adjacency list representation works better for most problems, for some problems with *dense* graphs the adjacency matrix is preferable.
- **Traversal**: There are many possible ways to explore a graph, but the most common are *Breadth First Search* (*BFS*) and *Depth First Search* (*DFS*). The former is better when we search for the path with the fewest edges between a source and a goal, while the latter is often used when we need to understand more about the structure of the graph: presence of cycles, connected and strongly connected components, topological order (for *directed acyclic graphs* only).
- **Minimum distance**: *Dijkstra's* algorithm extends BFS when the shortest path needs to be computed in terms of minimum edges' weight, not just minimum number of edges. It can be used when edges' weights are non-negative, and it's applied in many fields, including maps and IP routing.
- **Most efficient search**: *A** provides an improvement on Dijkstra when we have extra information besides the edges' weight, information that can be conveyed into a heuristic estimating the distance from each vertex to the goal. When this heuristic abides by certain properties, it can be proved that A* is optimal. This